Tutorial 2

I/O Streams review · Default params · Return semantics · Overloading

Three standard streams

- `cout` — write to **stdout**
- `cerr` — write to **stderr**
- `cin` — read from **stdin**

```
int x, y;
cin >> x >> y;        // read two ints
cout << x + y << endl;
```

- `>>` and `<<` are **overloaded** — direction of arrow = direction of data flow
- `cin >>` skips leading whitespace by default

Review: Stream failure & EOF


```
int i;
while (cin >> i) {          // implicit bool conversion
    cout << i << endl;
}
```

Condition	What to check
Bad input (not an int)	`cin.fail()` → `true`
End of file	`cin.eof()` → `true`

To **recover** from bad input (skip non-integers):

```
cin.clear();   // reset failure flags
cin.ignore();  // discard the offending character
```

Review: Strings & File Streams

`std::string` (import <string>)

- Grows automatically — no manual memory management
- `s.length()`, `s[i]`, `s1 + s2`, `s1 == s2`

File streams (import <fstream>)

```
ifstream f{ "file.txt" };   // opens file
string word;
while (f >> word) cout << word << endl;
// file closed automatically when f goes out of scope
```

String streams (import <sstream>)

```
istringstream iss{ someString };
int n;
if (iss >> n) { /* n is a valid int */ }
```

This week

1. **Default function parameters**
2. **Returning by value / pointer / reference**
3. **Function overloading**
4. **Operator overloading**

1 · Default Function Parameters


```
void printSuiteFile(string name = "suite.txt") {
    ifstream file{ name };
    for (string s; file >> s;)
        cout << s << endl;
}

printSuiteFile("suite2.txt"); // uses "suite2.txt"
printSuiteFile();             // uses "suite.txt"
```

- Optional params must appear **at the end** of the param list
- The **caller** supplies the default, not the function body

Why does the caller supply the default?

The call stack:

Normal call	caller pushes args → function reads from expected stack positions
Missing param	function has **no way to know** a param is absent — it reads garbage from the stack

So the **compiler** rewrites `printSuiteFile()` into `printSuiteFile("suite.txt")` at the call site.

Consequence: default arguments belong to the function's **interface**, not its implementation.

- In separate compilation → put defaults in the **header file**, not the `.cc` file.

2 · Returning: by Value


```
struct Node { int data; Node *next; };

Node getMeANode() {
    Node n;
    return n;   // n is copied to caller's stack frame
}
```

- Looks expensive — but the compiler usually elides the copy (**RVO / NRVO**)
- **Prefer return by value** unless you have a strong reason not to

2 · Returning: Dangling Pointer / Reference — BAD


```
Node* getMeANode() {    // ❌ BAD
    Node n;
    return &n;           // n is dead after return!
}

Node& getMeANode() {    // ❌ Also BAD
    Node n;
    return n;            // ref to dead local
}
```

Local variables live on the stack frame of the function. That frame is **gone** when the function returns → undefined behaviour.

```
Node* getMeANode() {   // ✅ OK, but...
    Node *np = new Node;
    return np;          // heap is alive after return
}
```

Heap data stays alive — but **you must `delete` it** when done.

Summary:

Strategy	Safe?	Notes
Return by value	✅	Preferred; compiler often elides copy
Return ptr to local	❌	Dangling pointer — UB
Return ref to local	❌	Dangling reference — UB
Return ptr to heap	✅	Safe, but caller must `delete`

C forces unique names for every function:

```
int  negInt (int  n) { return -n; }
bool negBool(bool b) { return !b; }
```

C++ lets functions **share a name** if their parameter lists differ:

```
int  neg(int  n) { return -n; }
bool neg(bool b) { return !b; }
```

- The compiler picks the right version at **compile time** based on argument types
- Overloads must differ in **number or types** of parameters
- Differing **only** in return type is **not enough**

```
neg(-3);    // int  arg → calls neg(int)
neg(true);  // bool arg → calls neg(bool)
```

You have already used overloading:

- `<<` behaves differently for `int`, `string`, …
- `>>` behaves differently for `int`, `string`, `istream`, …
- `+` adds numbers **or** concatenates strings

The compiler resolves the call at compile time — this is **static dispatch**.

You can give C++ operators meaning for your own types:

```
struct Vec { int x, y; };

Vec operator+(const Vec &v1, const Vec &v2) {
    return { v1.x + v2.x, v1.y + v2.y };
}
```

Usage:

```
Vec a{1, 2}, b{3, 4};
Vec c = a + b;   // calls operator+(a, b)  → {4, 6}
```

4 · Operator Overloading: Scalar Multiplication

```
Vec operator*(const int k, const Vec &v) {
    return { k * v.x, k * v.y };
}

Vec operator*(const Vec &v, const int k) {
    return k * v;   // reuse the one above!
}
```

- Define both orderings so `2 * v` and `v * 2` both work
- The second overload delegates to the first — less code to maintain

4 · Overloading << and >>

```
struct Grade { int theGrade; };

ostream &operator<<(ostream &out, const Grade &g) {
    out << g.theGrade << '%';
    return out;   // enables chaining: cout << g1 << g2
}

istream &operator>>(istream &in, Grade &g) {
    in >> g.theGrade;
    if (g.theGrade < 0)   g.theGrade = 0;
    if (g.theGrade > 100) g.theGrade = 100;
    return in;    // enables chaining: cin >> g1 >> g2
}
```

- Must return the stream by **reference** to support chaining
- `Grade` taken by `const &` in << (not modified), by `&` in >> (will be modified)

```
cout << g1 << g2 << endl;
```

is parsed left-to-right as:

```
((cout << g1) << g2) << endl
      ^
      returns cout (ostream&)
            ^
            returns cout again
```

Without returning `out`, you could not chain multiple << calls. Same reasoning applies to >>.

Common Pitfalls — Quick Reference

Mistake	**Fix**
Default param in the middle	Move optional params to the end
Return pointer/ref to local	Return by value, or `new` + document ownership
Overload differs only in return type	Change param types/count instead
<</>> overload returns `void`	Return `ostream&` / `istream&`
Forget `cin.clear()` after failure	Always `clear()` then `ignore()` to recover

This week's problem set


```
struct Party {
  string pokemon[6];
  int size = 0;
}
```

- `<<`: print size and contents of party to `ostream`
- `>>`: read in a single word (the name of a Pokémon) from `istream`, and add to the end of the party.
- `-`: given a party and index, returns a new party with Pokémon at index `i` removed and other Pokémon shifted down.

Assignment 1

Important Reminders:

- due date for part 1: Wednesday, Oct 7, 5 pm.
- due date for part 2: Wednesday, Oct 14, 5 pm.

Refer to Piazza first for questions and pull the most recent files.